



याद रखें / Remember

- * आपके खाते में नवीनतम केवायसी, नामांकन, मोबाइल नंबर एवं ई-मेल आईडी अद्यतन होने चाहिए.
Your account should be updated with your latest KYC, Nomination, Mobile number & Email ID.
- * अपनी पासबुक को सावधानी पूर्वक रखें. अपनी पासबुक पर हस्ताक्षर न करें. पासबुक को नियमित रूप से अद्यतन करें. किसी भी प्रकार की विसंगति होने की सूचना तत्काल दी जानी चाहिए.
Preserve your passbook carefully. Do not put your signature on passbook. Get passbook updated regularly. Any discrepancy should be notified immediately.
- * कृपया अपनी व्यक्तिगत जानकारी, प्रयोक्ता आईडी, पिन, पासवर्ड, सीवीवी संख्या किसी को न दें. किसी भी प्रकार की जानकारी मांगे जाने पर शाखा को सूचित किया जाना चाहिए.
Please do not give your personal information, user IDs, PINs, Password, CVV number to anyone. Any calls or e-mails requesting for such information should be notified to the branch.
- * खाते के लिए पहली पासबुक नि:शुल्क होगी. शुल्क का भुगतान किए जाने पर डुप्लीकेट पासबुक उपलब्ध करवायी जाएगी.
First passbook in account is free. Duplicate passbook may be provided on payment of fee.
- * प्रत्येक अर्धवर्ष में 50 डेबिट (स्थायी अनुदेशों, वैकल्पिक डिलिवरी चैनलों, सेवा प्रभारों से संबंधित प्रविष्टियों को छोड़कर) की अनुमति होगी. 50 प्रविष्टियों से अधिक लेन देन होने पर प्रभार लगाया जाएगा.
50 debits (excluding Standing Instruction, Alternate Delivery Channels, related to service charges) per half year is permitted. Transaction exceeding above 50 will be charged.
- * ब्याज की गणना दैनिक आधार पर की जाएगी और तिमाही की समाप्ति के 15 दिनों के भीतर खाते में जमा किया जाएगा.
Interest is calculated on daily basis and credited to accounts within 15 days of end of quarter.

32910100010444

Branch Name : ANGALAKUDURU, AP Phone : 08644-222688
Branch Address : MAIN ROAD
: ANGALAKUDURU P.O. TENALI (M) GUNTUR
Branch Email ID : angala@bankofbaroda.com CKYC NO : XXXXXXXXXXXX2922
MICR Code : 520012202 IFSC : BARBOANGALA [Fifth character is zero]
Customer ID : 05C068018 PPD NO : NA
Account Number : 32910100010444 A/c Opening Date : 18-11-2023
Account Name : AKULA SUJAN
Address : S/O MADHAVA RAO CHAVA VARI PALEM TENALI
: MANDAL ANGALAKUDURU 13-239 GUNTUR
: GUNTUR AP 522211
Nominee Name : A SRI NAGA LAKSHMI



VIGNAN'S LARA
INSTITUTE OF TECHNOLOGY & SCIENCE

Vadlamudi, Guntur Dist (AP) - 522 213, Ph: 2381200, Fax : 2381239

STUDENT IDENTITY CARD

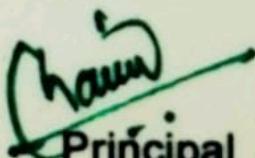


AKULA SUJANA

B.Tech. CSE - AI

22FE1A4302

2022 - 2026


Principal

Father Name AKULA MADHAVA RAO

Address 13-239, Near shivalayam, Chavavaripalem, TENALI,
Guntur, A.P - 522211

Phone No 8121223075

Aadhar No. 6743 2522 7711

www.vignanlara.org

If found Pleasa return to

VIGNAN'S LARA INSTITUTE OF TECHNOLOGY & SCIENCE

Vadlamudi post, Guntur District - 522213 Tel: 08632381200



22FE1A4302

INTERNSHIP FINAL REPORT

DEVELOPMENT AND EVALUATION OF A RETRIEVAL-AUGMENTED GENERATION SYSTEM FOR GST DOCUMENTS

Internship Duration
11 May 2026 – 10 July 2026

Name : Akula Sujan

ABSTRACT:

This report presents the work carried out during an internship from 18 May 2026 to 17 July 2026. The internship began with strengthening Python programming, Object-Oriented Programming and logical problem-solving skills. This was followed by a theoretical study of Generative AI, Large Language Models, text embeddings, vector databases, semantic search and Retrieval-Augmented Generation.

The major project undertaken during the internship was the development of a Retrieval-Augmented Generation system for GST legal documents. A large GST Acts and Rules PDF was processed, divided into document chunks, converted into vector embeddings and stored in ChromaDB. A semantic retriever was implemented to identify relevant document chunks for user queries, and the retrieved context was supplied to a Large Language Model for answer generation.

The project was developed iteratively. The initial implementation experienced duplicate document retrieval and achieved approximately 45% manually evaluated retrieval accuracy. After debugging repeated ChromaDB insertion and rebuilding the vector collection, retrieval consistency improved and accuracy increased to approximately 70%. Further experiments with chunking strategies and embedding models were conducted. The final system used sentence-transformers/all-MiniLM-L12-v2, ChromaDB, a chunk size of 500, chunk overlap of 100 and Top-3 semantic retrieval, achieving approximately 85% manual retrieval accuracy.

1. Introduction
2. Objectives of the Internship
3. Weekly Progress and Work Completed
4. Project Architecture
5. Technologies and Tools Used
6. Challenges Faced
7. Key Learning Outcomes
8. Results and Evaluation
9. Conclusion

1. INTRODUCTION

During my internship from 18 May 2026 to 17 July 2026, I focused on strengthening my programming skills and learning concepts in Machine Learning, Natural Language Processing, Generative AI, and Retrieval-Augmented Generation (RAG).

The major objective was to develop a RAG-based system capable of retrieving relevant information from a large GST legal document and generating context-aware answers. The project was developed and improved through continuous testing, debugging, chunking experiments, and embedding model evaluation.

The final system was enhanced with Cross-Encoder reranking and a Streamlit-based conversational interface, transforming the project into an interactive GST AI Assistant with semantic retrieval and LLM-based answer generation.

2. OBJECTIVES OF THE INTERNSHIP

- Strengthen Python programming and Object-Oriented Programming fundamentals.
- Improve logical and problem-solving skills through competitive programming platforms.
- Understand fundamental concepts of Machine Learning, Deep Learning, and Natural Language Processing.
- Understand Generative AI, Large Language Models, and Retrieval-Augmented Generation.
- Learn document loading, text chunking, embeddings, vector databases, semantic search, and reranking.
- Develop a practical GST document question-answering system using RAG architecture.
- Experiment with chunking configurations and embedding models to improve retrieval quality.
- Improve retrieval relevance using Cross-Encoder reranking.
- Develop an interactive conversational interface using Streamlit.
- Identify, debug, test, and evaluate the developed RAG system using GST-related questions.

3. WEEKLY PROGRESS AND WORK COMPLETED

WEEK 1: PYTHON, OBJECT-ORIENTED PROGRAMMING AND PROBLEM SOLVING

During the first week of the internship, I focused on strengthening my programming fundamentals using Python.

I revised important Python concepts such as variables, data types, conditional statements, loops, functions, lists, tuples, dictionaries, sets, file handling, exception handling, and modules.

I also studied Object-Oriented Programming concepts including classes and objects, constructors, inheritance, encapsulation, abstraction, and polymorphism.

Along with theoretical learning, I practiced programming and logical problem-solving questions on platforms such as LeetCode, CodeChef, and Codeforces.

The main purpose of this week was to improve logical thinking and develop the programming skills required for implementing the project.

Key Learning Outcomes:

- Improved Python programming skills.
- Better understanding of Object-Oriented Programming.
- Stronger logical problem-solving ability.
- Increased familiarity with coding platforms.

WEEK 2: INTRODUCTION TO GENERATIVE AI AND RETRIEVAL-AUGMENTED GENERATION

During the second week, I studied the theoretical concepts required to understand Retrieval-Augmented Generation.

I learned about Large Language Models and how they generate responses based on learned language patterns. I also studied some of the limitations of standalone LLMs, particularly hallucination and the inability to directly access domain-specific documents that were not part of their available context.

Retrieval-Augmented Generation was studied as an approach that combines an information retrieval system with a Large Language Model.

The basic RAG workflow studied during this week was:

Document → Text Extraction → Text Chunking → Embedding Generation → Vector Database → Query Embedding → Similarity Search → Relevant Context Retrieval → LLM Answer Generation

I learned the concepts of text embeddings, vector representations, cosine similarity, semantic search, vector databases, document chunking, chunk overlap, retrievers, and prompt-based context injection.

This theoretical understanding provided the foundation for developing the GST RAG project during the following weeks.

WEEK 3: INITIAL DEVELOPMENT OF THE GST RAG SYSTEM

From the third week onwards, I started the practical development of the RAG system. A GST Acts and Rules PDF containing approximately 438 pages was selected as the knowledge source for the project.

The initial RAG architecture consisted of a PDF document loader, recursive text splitter, embedding model, ChromaDB vector database, semantic retriever, and a Large Language Model.

The GST PDF was loaded using PyMuPDFLoader. The extracted document content was divided into smaller chunks using RecursiveCharacterTextSplitter.

Initially, the all-MiniLM-L6-v2 Sentence Transformer model was used to generate embeddings for the document chunks. The generated embeddings and document metadata were stored in ChromaDB. A custom retrieval class was developed to generate an embedding for the user query and retrieve the Top-K semantically similar document chunks from the vector database.

The first version of the system used all-MiniLM-L6-v2, ChromaDB, and Google Gemma through OpenRouter. The project involved implementing document loading, text chunking, embedding generation, vector storage, and Top-K semantic retrieval as separate components.

Along with the RAG development, I started learning the fundamentals of **Machine Learning**. I studied the different types of Machine Learning, including **Supervised Learning, Unsupervised Learning, and Reinforcement Learning**. I learned about features, target variables, training data, testing data, and the basic process of training and evaluating a Machine Learning model. I studied **Linear Regression** and understood its use in predicting continuous numerical values. I also learned **Logistic Regression** and its application in binary classification problems. The concepts of **regression and classification** helped me understand how Machine Learning algorithms are selected based on the type of problem.

I also learned the basic Machine Learning workflow, including **data collection, data preprocessing, handling missing values, feature selection, train-test splitting, model training, prediction, and model evaluation**.

WEEK 4: RAG INTEGRATION, TESTING AND IDENTIFICATION OF RETRIEVAL PROBLEMS

During the fourth week, the output of the retrieval system was integrated with a Large Language Model. The retrieved document chunks were combined to create a context, and the context along with the user query was passed to the LLM through a structured prompt.

The system was tested using different groups of GST-related questions to evaluate the retrieval performance under different conditions.

Group A – Definition Retrieval:

- What is works contract?
- What is voucher?
- What is union territory tax?
- What is aggregate turnover?
- What is business?
- What is consideration?
- What is goods?

- What is services?
- What is input tax credit?
- What is principal supply?

Group B – Clause Lookup:

- What is defined under Section 2(119)?
- What is defined under Section 2(118)?
- What is defined under Section 2(115)?
- What is defined under Section 2(102)?
- What is defined under Section 2(17)?

Group C – Section Retrieval:

- Who appoints officers under GST?
- What are the powers of officers under Section 5?
- What does Section 6 deal with?
- What is assessment?
- What is audit?

Group D – Similar-Term Confusion Test:

- What is input?
- What is input service?
- What is input tax?
- What is input tax credit?

Group E – Hallucination Resistance Test:

- What is GST SPL-05?
- What is FORM GST SPL-05 about?
- What is online money gaming?
- What is virtual digital asset?
- What is specified actionable claim?

Group F – Negative Test:

- What is GST 2.0?
- What is Artificial Intelligence under GST?
- What is cryptocurrency tax under Section 500?
- What is blockchain under CGST Act?
- What is GST metaverse registration?

During testing, a major retrieval problem was identified. The system frequently returned the same document chunks multiple times. Further analysis showed that document embeddings were repeatedly inserted into the persistent ChromaDB collection whenever the program was executed.

The vector database contained approximately **23,065 entries**, whereas only around **3,740 document chunks** were expected. As a result, identical chunks were retrieved repeatedly, and the initial manually evaluated retrieval accuracy was approximately **45%**.

This week was particularly useful in understanding that the performance of a RAG system depends not only on the Large Language Model but also on the **quality, correctness, and reliability of the retrieval pipeline**.

WEEK 5: DEBUGGING CHROMADB AND IMPROVING RETRIEVAL CONSISTENCY

During the fifth week, I focused on identifying and resolving the duplicate retrieval problem.

The existing ChromaDB collection was deleted, and the vector database was rebuilt from scratch.

The initial implementation used randomly generated UUID-based document identifiers. This was replaced with deterministic document IDs such as doc_0, doc_1, and doc_2.

The vector store initialization process was modified to prevent repeated insertion of identical document embeddings.

After rebuilding the database, the collection count was reduced from approximately 23,065 entries to 3,740 entries, matching the expected number of chunks. The duplicate retrieval problem disappeared, and the retrieval consistency improved significantly.

The manually evaluated retrieval accuracy increased from approximately 45% to 70%. During this stage, I learned the importance of vector database management, deterministic document identifiers, persistent storage handling, and debugging retrieval pipelines.

Along with the project development, I continued learning different **Machine Learning algorithms**. I studied **Decision Trees** and understood how decisions are made using feature-based conditions. I also learned about **Random Forest**, which combines multiple decision trees to improve prediction performance and reduce overfitting.

I studied **K-Nearest Neighbors (KNN)** and understood how similarity or distance between data points can be used for classification and prediction. I also gained an understanding of **Support Vector Machines (SVM)** and how a decision boundary or hyperplane is used to separate different classes. I further learned about **Naive Bayes**, its probability-based approach to classification, and basic **clustering techniques such as K-Means Clustering** for grouping similar data points in unsupervised learning.

During this week, I also studied **overfitting and underfitting, bias and variance, and the bias-variance trade-off**. These concepts helped me understand the importance of developing models that perform well on both training data and unseen data.

WEEK 6: EXPERIMENTATION WITH TEXT CHUNKING STRATEGIES

During the sixth week, I experimented with different text chunking configurations to determine whether retrieval quality could be improved.

Different chunk sizes and chunk overlaps were evaluated. The initial system used larger chunks with configurations such as a chunk size of 1000 and an overlap of 200.

Smaller chunk configurations were later tested to provide more focused semantic information to the embedding model.

I also experimented with chapter-based and section-based separators because GST legal documents are naturally structured into chapters, sections, and rules.

However, the experiment showed that chapter and section-based splitting did not produce a significant improvement in retrieval quality.

In some cases, the system retrieved chapter headings rather than the detailed legal content required to answer the question.

Based on these observations, the separator modifications were reverted.

The final chunking configuration used a chunk size of 500 and a chunk overlap of 100.

The experiments indicated that the chunking strategy was not the primary retrieval bottleneck.

This experimentation helped me understand the effect of chunk size, chunk overlap, and document structure on semantic retrieval.

Along with the RAG experiments, I learned important concepts related to **Machine Learning model training and optimization**. I studied **loss functions and cost functions** and understood how they are used to measure the difference between predicted and actual values.

I learned the concept of **Gradient Descent** and how model parameters are updated iteratively to minimize the loss function. I also studied the importance of the **learning rate** and its effect on model convergence.

I learned about **feature scaling, normalization, and standardization** and understood their importance when working with algorithms that depend on the distance or magnitude of features.

I also studied **regularization techniques such as L1 and L2 regularization**, which are used to reduce overfitting and improve model generalization.

In addition, I learned about **cross-validation and hyperparameter tuning** and understood how they are used to evaluate and improve Machine Learning models.

I studied common evaluation metrics such as **accuracy, precision, recall, F1-score, confusion matrix, Mean Absolute Error, Mean Squared Error, and R-squared score**. These concepts helped me understand how classification and regression models are evaluated using different performance measures.

WEEK 7: EMBEDDING MODEL EVALUATION AND PERFORMANCE COMPARISON

During the seventh week, I focused on improving retrieval quality by experimenting with different embedding models. The initial all-MiniLM-L6-v2 model was compared with intfloat/e5-base-v2. The E5 model provided noticeably better retrieval results for several GST-related queries. However, the embedding generation process became significantly slower. For the approximately 450-page GST PDF, MiniLM generated embeddings in less than five minutes, whereas e5-base-v2 required approximately 48 minutes.

Although the E5 model improved retrieval quality, the indexing time made repeated development and testing impractical. I then evaluated sentence-transformers/all-MiniLM-L12-v2. The L12 model retrieved more relevant chunks for GST definitions, registration, returns, and assessment-related queries. It also maintained a more reasonable embedding generation time compared with E5.

The manually evaluated retrieval results were:

Embedding Model	Approximate Retrieval Accuracy
all-MiniLM-L6-v2	77%
all-MiniLM-L12-v2	85%
intfloat/e5-base-v2	89%

The all-MiniLM-L12-v2 model improved retrieval accuracy by approximately 8% compared with the L6 model while requiring significantly less embedding time than E5. Therefore, all-MiniLM-L12-v2 was selected as the final embedding model.

This experiment helped me understand the practical trade-off between model accuracy, computational requirements, indexing time, and development efficiency. Along with the embedding model experiments, I learned the fundamentals of **Natural Language Processing (NLP)** and understood how text data is processed and represented for Machine Learning and Deep Learning applications. I studied concepts such as **text preprocessing, tokenization, word representations, embeddings, and semantic similarity**.

I also learned about **Multi-Layer Perceptrons (MLP)** and the basic structure of neural networks consisting of **input layers, hidden layers, and output layers**. I studied the concepts of **forward propagation and backpropagation** and understood how neural networks calculate predictions and update their weights based on errors. I further learned about **Gradient Descent** and how it is used to minimize the loss function by updating neural network parameters during training.

I studied different **activation functions**, including **ReLU, Leaky ReLU, Sigmoid, and Softmax**, and understood their roles in introducing non-linearity and producing suitable outputs for different neural network tasks. I also learned about the **vanishing gradient and exploding gradient problems**, which can occur during backpropagation in deep neural networks. These concepts helped me understand the challenges involved in training deeper neural networks and the importance of activation functions and proper parameter

initialization. Overall, this week improved my understanding of **NLP, text embeddings, neural networks, Multi-Layer Perceptrons, backpropagation, Gradient Descent, activation functions, and common gradient-related problems**, along with the practical evaluation of embedding models for the GST RAG system.

WEEK 8: FINAL RAG ENHANCEMENT, STREAMLIT INTERFACE, TESTING AND PROJECT DOCUMENTATION

During the final stage of the internship, I focused on improving the usability and retrieval performance of the GST RAG system. A web-based user interface was developed using Streamlit to provide a more interactive and user-friendly experience. The initial query-based system was enhanced into a chat-based conversational interface. Session state was used to maintain the conversation history so that previous user queries and generated answers remained visible while asking new questions. This provided a better interaction experience similar to modern AI-based chat applications.

The application was also enhanced with options to select different Large Language Models and embedding models through the sidebar. This made it easier to experiment with different models and compare their performance without modifying the system manually. To improve retrieval quality, a Cross-Encoder reranking model was integrated into the RAG pipeline. The semantic retriever first retrieved relevant document chunks, and the reranking model reordered the retrieved passages based on their relevance to the user query. The highest-ranked passages were then used as context for answer generation.

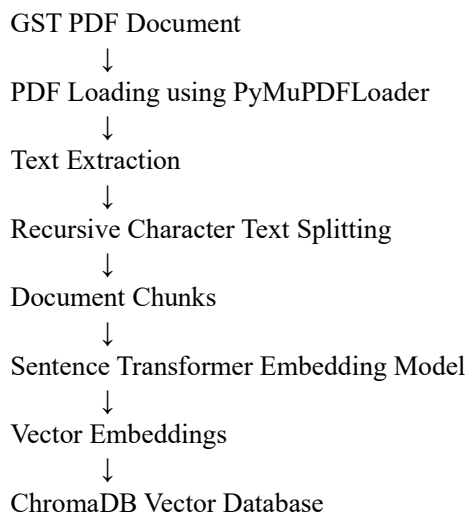
The interface also displayed the retrieved passages along with their reranking scores, similarity scores, and document metadata. Resource caching, error handling, and loading indicators were added to improve application performance and user experience. The improved RAG system was further evaluated using a manually prepared benchmark of GST-related questions. The retrieved chunks were examined for correctness and relevance, and the generated answers were cross-checked using the retrieved context.

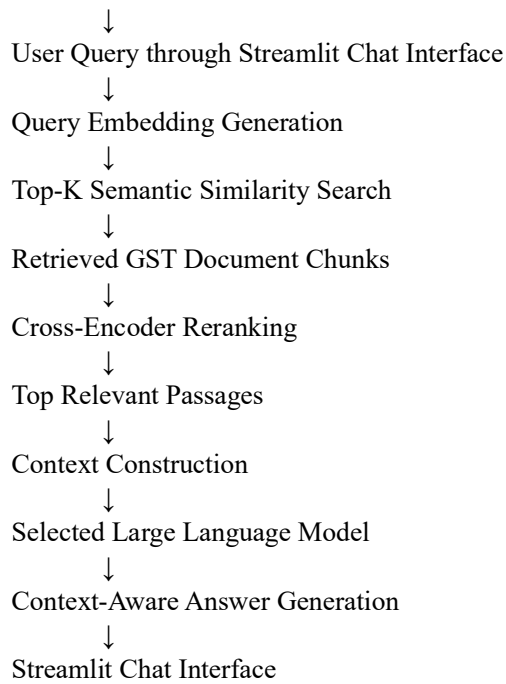
The final system used sentence-transformers/all-MiniLM-L12-v2, ChromaDB, semantic retrieval, Cross-Encoder reranking, Large Language Model integration, and a Streamlit-based conversational interface.

The final week also involved organizing the experimental observations, documenting different project versions, recording problems encountered during development, and summarizing the conclusions obtained from each experiment.

Overall, the internship provided practical experience in Python programming, Object-Oriented Programming, Machine Learning, Deep Learning, Natural Language Processing, Generative AI, Large Language Models, Retrieval-Augmented Generation, text embeddings, vector databases, semantic retrieval, reranking, Streamlit application development, debugging, model evaluation, and the development of an end-to-end RAG-based application.

4. PROJECT ARCHITECTURE





5. TECHNOLOGIES AND TOOLS USED

Category	Technology / Tool	Purpose
Programming	Python	Core project implementation
User Interface	Streamlit	Interactive conversational web interface
Document Processing	PyMuPDFLoader	Loading GST PDF documents
Text Processing	RecursiveCharacterTextSplitter	Document chunking
Embeddings	Sentence Transformers	Semantic vector generation
Embedding Models	MiniLM-L6-v2, MiniLM-L12-v2, E5-base-v2	Embedding and retrieval experiments
Vector Database	ChromaDB	Embedding storage and semantic similarity search
Reranking	Cross-Encoder / MS MARCO MiniLM	Reranking retrieved document passages
Large Language Models	Llama, GPT-OSS, Gemma, Nemotron	Context-aware answer generation and model experimentation
LLM Access	OpenRouter	Accessing different Large Language Models
Application Management	Streamlit Session State and Caching	Conversation history and model resource management
Problem Solving	LeetCode, CodeChef, Codeforces	Logical and programming practice

6. CHALLENGES FACED

Duplicate Retrieval

Repeated insertion of embeddings into persistent ChromaDB caused identical chunks to appear multiple times and increased the collection beyond the expected chunk count.

Retrieval from Legal Documents

GST documents contain repeated legal terminology, sections, rules, amendments, and forms, making accurate semantic retrieval challenging.

Chunking Strategy

Large chunks sometimes contained excessive information, while structure-based splitting occasionally retrieved headings without sufficient legal context.

Embedding Model Selection

Higher-quality embedding models improved retrieval performance but increased indexing time and computational requirements.

Retrieval Ranking

Semantic similarity search sometimes retrieved related but less relevant passages. Cross-Encoder reranking was integrated to improve the ordering and relevance of retrieved chunks.

Application Performance and User Experience

Maintaining conversation history and avoiding repeated model loading were important challenges while developing the Streamlit interface. Session state and resource caching were used to improve application performance and user experience.

Evaluation

The project required a manually prepared GST question benchmark and manual relevance checking because a standardized automated RAG evaluation benchmark was not used.

7. KEY LEARNING OUTCOMES

The internship provided practical experience in Python, Object-Oriented Programming, Machine Learning, Deep Learning, Natural Language Processing, and AI application development. I developed a strong understanding of the complete RAG pipeline and how document processing, embeddings, vector databases, semantic retrieval, reranking, and Large Language Models interact.

A major learning was that retrieval quality and ranking are critical to a RAG system. I gained practical knowledge of ChromaDB management, embedding model comparison, Cross-Encoder reranking, manual evaluation, and performance trade-offs.

I also gained experience in developing an interactive Streamlit-based conversational application, maintaining conversation history, caching model resources, and improving user experience.

Most importantly, I learned to follow an iterative engineering approach: identify a problem, implement improvements, test the system, evaluate the results, and document the conclusions.

8. RESULTS AND EVALUATION

Stage	Major Change	Approx. Manual Retrieval Accuracy
Initial Version	Basic RAG pipeline; duplicate vector entries	45%
Version 1.1	Rebuilt ChromaDB and removed duplicate insertions	70%
MiniLM-L6 Evaluation	Improved pipeline and benchmark evaluation	77%
MiniLM-L12 Evaluation	Higher-quality MiniLM embeddings	85%

E5-base-v2 Experiment	Improved retrieval but very slow indexing	89%
-----------------------	---	-----

The final selected configuration used sentence-transformers/all-MiniLM-L12-v2, ChromaDB, semantic similarity retrieval, and Cross-Encoder reranking. The MiniLM-L12 model achieved approximately 85% manual retrieval accuracy and provided a better balance between retrieval quality and embedding generation time. The final RAG pipeline was further enhanced with Cross-Encoder reranking, which reordered the retrieved passages based on their relevance to the user query before context construction. A Streamlit-based conversational interface was also integrated to provide model selection, conversation history, and an improved user experience.

9. CONCLUSION

The internship provided an opportunity to move from theoretical learning to the practical development of an Artificial Intelligence application. It started with Python, Object-Oriented Programming, Machine Learning, Deep Learning, and Natural Language Processing and progressed to Generative AI and Retrieval-Augmented Generation. The main project involved developing a GST-based RAG system capable of processing a large legal document, generating embeddings, storing them in a vector database, retrieving relevant information, reranking retrieved passages, and generating context-aware answers using Large Language Models.

During development, multiple challenges were identified and resolved through systematic experimentation. Duplicate retrieval was fixed, different chunking strategies and embedding models were evaluated, and Cross-Encoder reranking was integrated to improve passage relevance. The final system used all-MiniLM-L12-v2, ChromaDB, semantic retrieval, Cross-Encoder reranking, and a Streamlit-based conversational interface. Overall, the internship significantly improved my understanding of Machine Learning, NLP, Generative AI, semantic search, vector databases, embedding models, RAG, reranking, and AI application development. It also provided valuable experience in developing, debugging, evaluating, and documenting a complete end-to-end AI-based application.